NLP ASSIGNMENT — WEEK 1 · RECURRENT NEURAL NETWORKS

Toxic-Query Classification with an LSTM

A gated recurrent network for content-safety triage

Notebook	`lstm.ipynb`
Task	Multi-class toxic-content classification (9 categories) using an LSTM
Dataset	`cellula_toxic_data.csv` (3,000 raw rows)
Author	Eslam Mohamed Fawzy
Date	9 July 2026
Framework	TensorFlow / Keras, scikit-learn, NLTK

Abstract

This notebook re-implements the toxic-query classifier of the companion RNN notebook, replacing the vanilla `SimpleRNN` with a gated **Long Short-Term Memory (LSTM)** layer and adding a hidden `Dense` layer with `Dropout` regularisation. The preprocessing pipeline is identical: clean → tokenise → pad → stratified split. The LSTM is designed to overcome the vanishing-gradient weakness of a simple RNN. On this dataset it reaches a **weighted F1 of 0.717** (macro 0.310), which is *below* the assignment target of F1 ≥ 0.85; the report analyses why, honestly, and prescribes concrete fixes.

1 Objective

The notebook trains a supervised **multi-class text classifier** that assigns each short user query to one of nine **content-safety categories** (*Safe, Violent Crimes, Non-Violent Crimes, unsafe, Unknown S-Type, Suicide & Self-Harm, Elections, Sex-Related Crimes, Child Sexual Exploitation*). It is the second model in a two-part study: where the first notebook used a `SimpleRNN`, this one uses an **LSTM** — the canonical fix for the memory limitations of vanilla recurrent networks.

Why the task matters

A content-safety classifier acts as a guardrail in front of an AI assistant: it flags harmful requests (violence, self-harm, exploitation) so they can be blocked or escalated before a model responds. Catching the harmful minority classes is the whole point, which makes **recall on rare classes** — and therefore the **F1-score** — the metric that actually matters.

NLP & deep-learning concepts exercised

Concept	Where used
Text normalisation & cleaning	lower-case, contraction expansion, punctuation/URL removal
Tokenisation + OOV vocabulary	Keras `Tokenizer`
Padding / truncation	fixed length 21 (95th percentile)
Trainable word embeddings	128-d `Embedding` layer
Gated sequence modelling	`LSTM(64)` with input/forget/output gates
Regularisation	`Dropout(0.5)` after a hidden dense layer
Imbalanced-class evaluation	weighted & macro F1, per-class report

2 Dataset Description

Identical source to the RNN notebook: `Data/cellula_toxic_data.csv`, three columns.

Column	Meaning	Distinct (raw)
`query`	user request — the primary signal	2,009 / 3,000
`image descriptions`	short image caption (12 fixed templates)	12
`Toxic Category`	**target**, 9 classes	9

- **Raw:** 3,000 rows; **no missing values.**
- **Duplicates:** 973 removed → **2,027 unique rows.**

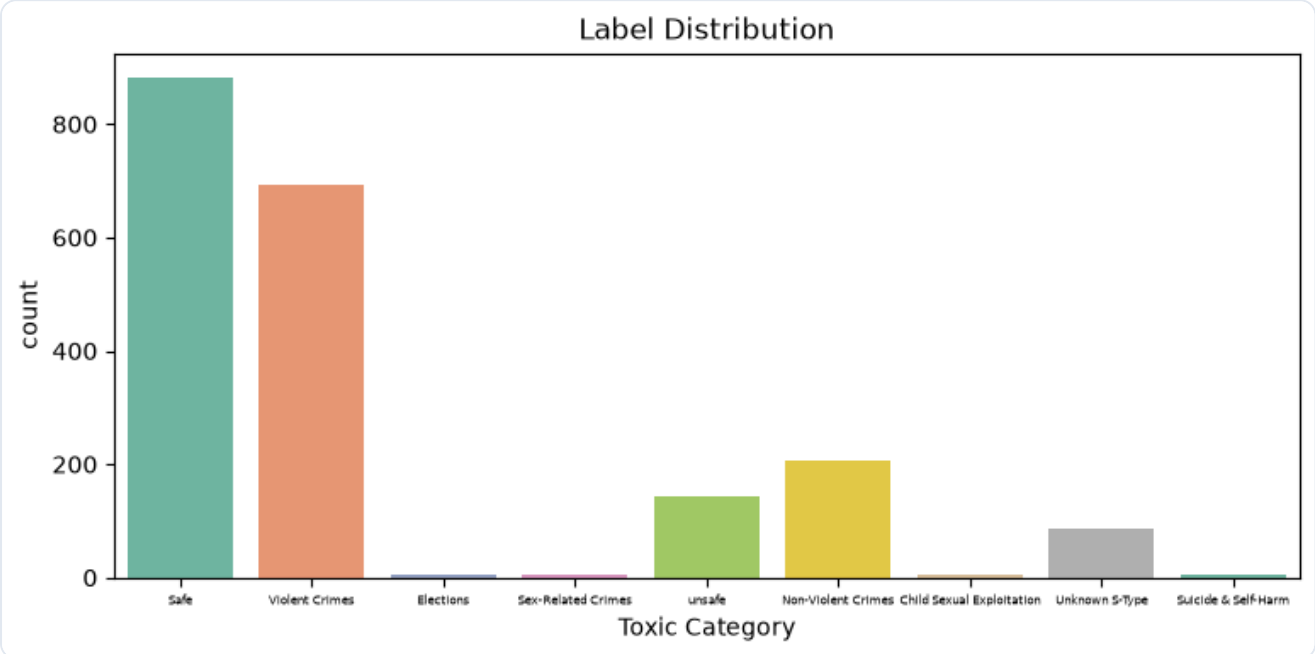
Class	Count	Share
Safe	881	43.5 %
Violent Crimes	693	34.2 %
Non-Violent Crimes	207	10.2 %
unsafe	143	7.1 %
Unknown S-Type	86	4.2 %
Suicide & Self-Harm / Elections / Sex-Related / Child Sexual Exploitation	5 / 4 / 4 / 4	<0.3 % each

SEVERE CLASS IMBALANCE

Two classes hold ~78 % of the data; four classes have ≤5 examples. This imbalance is the dominant factor behind the results in §9 and is why the F1-score (not accuracy) is the assignment's target metric.

3 Exploratory Data Analysis (EDA)

3.1 Label distribution



Examples per Toxic Category after de-duplication (notebook `sns.countplot`).

What / why / conclusion: the plot shows two dominant bars and five near-invisible ones. It confirms that the model will be data-starved on the minority classes, foreshadowing the low macro-F1 in §9.

3.2 Text length & vocabulary

Field	Mean words	Median	Max

`query`	13.7	12	140
`image descriptions`	8.2	8	11
`combined`	22.9	21	152

After cleaning, the tokeniser learns **4,500 unique words** (`vocab_size = 4,501`). The token-sequence lengths have a median of 10 and a 95th percentile of 21, motivating `maxlen = 21`.

4 Data Preprocessing

The pipeline is **identical** to the RNN notebook, so it is summarised here; the reasoning for each step is the same NLP logic (normalise → encode → fix length).

#	Step	What & why	Result
1	Drop duplicates	avoid train/test leakage & biased frequencies	3,000 → 2,027
2	V-stack query + caption	augment data; caption as weak label signal	2,027 → 4,054
3	`clean_text()`	lower-case, expand contractions, strip URLs/HTML/punctuation, collapse spaces	normalised text
4	Tokenise (+ `<OOV>`)	words → frequency-ranked integer ids; unseen words → OOV	vocab 4,501
5	Pad/truncate `maxlen=21`	fixed-length tensor; 95 % of sentences kept whole	`X` = (4054, 21)
6	`LabelEncoder`	9 label strings → ints 0–8 (needed by sparse-CE loss)	`y` ∈ {0..8}
7	Stratified 80/20 split	preserve class ratios across splits	3,243 train / 811 test

WHY MAXLEN = 21

The 95th percentile of token-sequence lengths is 21, so this length keeps 95 % of inputs intact while truncating only the extreme long tail (up to 147 tokens) — a balanced coverage-vs-efficiency choice.

CLEANING VERIFIED

The notebook prints five before/after pairs, e.g. *"What's the best way…?"* → *"what is the best way…"* — lower-cased, contraction expanded, punctuation removed.

5 Feature Engineering — Word Embeddings

Rather than sparse Bag-of-Words / TF-IDF vectors, the model learns a **dense 128-dimensional embedding** per word.

Method	Pros	Cons
Bag-of-Words / TF-IDF	simple, interpretable	sparse, ignores word order & semantics
Trainable Embedding (used)	dense, semantic, order-preserving via the LSTM	needs data; less interpretable

> **INTUITION**
> The embedding is a learnable (4501 × 128) matrix; token id t selects row E_t. Training pulls semantically similar toxic words together in vector space, giving the LSTM a meaningful signal to reason over.

6 Model Architecture

```
from tensorflow.keras.layers import LSTM, Embedding, Dense, Dropout
model = Sequential([
    Embedding(vocab_size, 128, input_length=21),
    LSTM(64),
    Dense(32, activation="relu"),
    Dropout(0.5),
    Dense(num_classes, activation="softmax")])
```

Parameter counts are **derived** (the notebook does not call `model.summary()`).

Layer	Output shape	Params	Role
`Embedding(4501,128)`	(21, 128)	576,128	token id → 128-d vector
`LSTM(64)`	(64)	49,408	gated sequence reader → 64-d summary
`Dense(32, relu)`	(32)	2,080	non-linear feature extraction
`Dropout(0.5)`	(32)	0	regularisation — randomly zeroes half the units
`Dense(9, softmax)`	(9)	297	class-probability output
Total trainable		**627,913**	—

6.1 Embedding

Input dim 4,501, output dim 128 — converts the (batch, 21) integer matrix into (batch, 21, 128) real vectors. It owns 92 % of the model's parameters.

6.2 LSTM (64 units) — why it beats a vanilla RNN

An LSTM cell maintains, in addition to a hidden state h_t, a **cell state** C_t regulated by three gates that decide what to forget, what to store, and what to output:

$$f_t = \sigma(\ldots) \quad i_t = \sigma(\ldots) \quad o_t = \sigma(\ldots) \quad C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

The **forget gate** f_t lets gradient flow across many steps almost unchanged, which **solves the vanishing-gradient problem** that limits a `SimpleRNN`. With 64 units it produces a 64-d sentence summary. The gating is why an LSTM has ~4× the parameters of a same-width simple RNN (four weight sets instead of one): *4×64×(64+128+1)=49,408*.

> **WHY THIS LAYER WAS CHOSEN**
>
> Toxic intent can hinge on a word early in a sentence ("*how to build a* … bomb"). The LSTM's long-term memory is meant to retain such early signals better than a simple RNN — the central hypothesis of this notebook.

6.3 Dense(32, ReLU)

A hidden fully-connected layer that lets the network combine the 64 LSTM features non-linearly before classification. **ReLU** (*max(0,x)*) is a cheap, gradient-friendly activation.

6.4 Dropout(0.5)

> **REGULARISATION ADDED HERE (VS THE RNN NOTEBOOK)**
>
> During training, `Dropout(0.5)` randomly sets 50 % of the 32 dense activations to zero on each step. This prevents the network from relying on any single feature and combats over-fitting — a direct improvement over the RNN model, which had no dropout. (At inference dropout is disabled.)

6.5 Output Dense(9) + Softmax

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Nine outputs that sum to 1; the predicted category is the `argmax`.

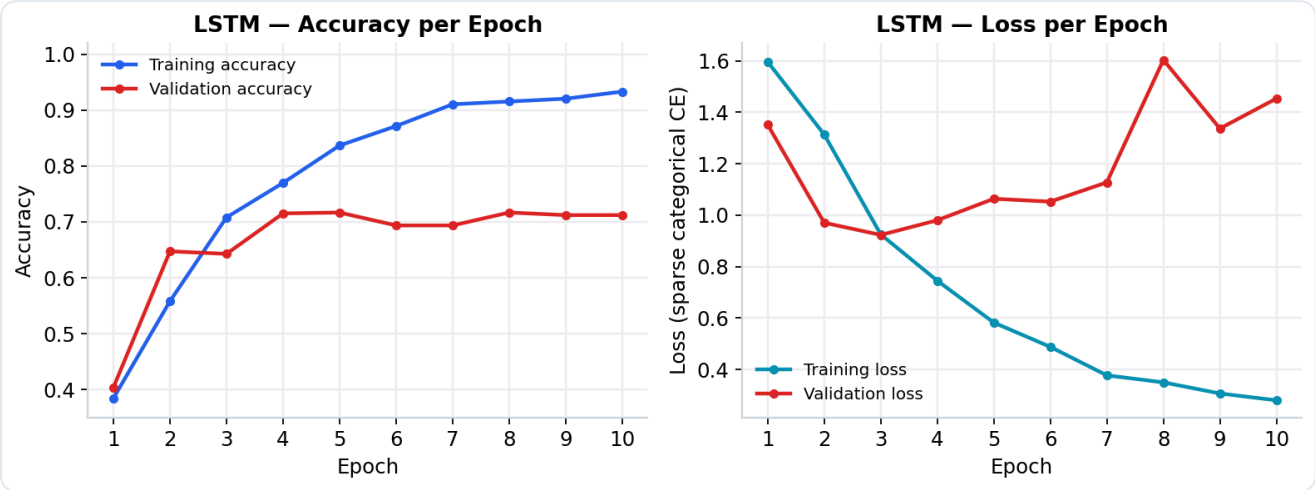
7 Training Process

```
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
history = model.fit(X_train, y_train, validation_split=0.2,
                    epochs=10, batch_size=32)
```

Setting	Value	Why appropriate
Optimiser	adam	adaptive, strong default

Loss	sparse categorical CE	multi-class, integer labels
Batch size	32	stable small-batch gradients
Epochs	10	enough to converge on this small set
Validation split	0.2	649 rows held out to watch generalisation
Callbacks / early stopping	none	a gap — would help pick the best epoch (val was best around epoch 4–5)

8 Training Results



LSTM training vs validation accuracy (left) and loss (right) per epoch, reconstructed from the notebook's training log.

Epoch	1	3	5	7	10
Train accuracy	0.38	0.71	0.84	0.91	**0.93**
Val accuracy	0.40	0.64	0.72	0.69	**0.71**
Train loss	1.60	0.92	0.58	0.38	**0.28**
Val loss	1.35	0.92	1.06	1.13	**1.45**

DIAGNOSIS: SLOWER, HEALTHIER START — THEN OVER-FITS

Compared with the SimpleRNN, the LSTM *learns more gradually* (train accuracy rises from 0.38, not a sudden jump), which is the expected behaviour of a gated network and evidence the dropout is working early on. Validation accuracy peaks around **0.72 at epoch 5**, then the model begins to over-fit: validation loss turns upward (0.92 → 1.45) while training loss keeps falling. Best generalisation is mid-training, again arguing for **early stopping**.

9 Evaluation Metrics

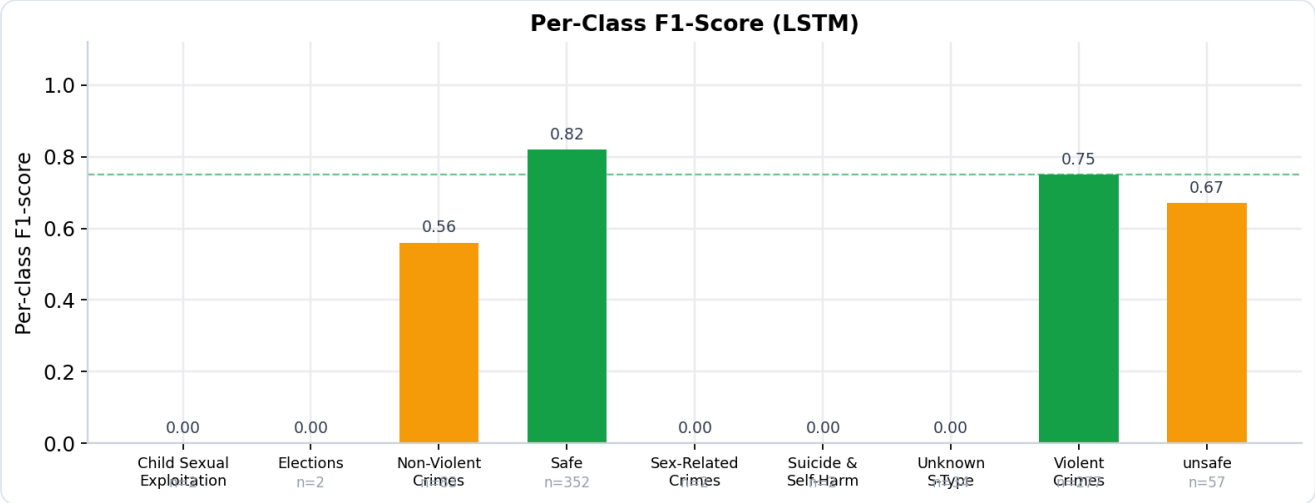
0.735	0.717	0.310	1.234
ACCURACY	WEIGHTED F1	MACRO F1	TEST LOSS

9.1 The F1-score and why it is the right metric

$$\text{Precision} = \frac{\text{TP}}{\text{TP+FP}} \qquad \text{Recall} = \frac{\text{TP}}{\text{TP+FN}} \qquad \text{F1} = 2\cdot\frac{\text{Precision}\cdot\text{Recall}}{\text{Precision+Recall}}$$

F1 is the harmonic mean of precision and recall — a class scores well only if it is both found and found accurately. On this ~78%-majority dataset, accuracy would flatter a lazy model, so **weighted F1** (size-weighted average) and **macro F1** (equal-weight average, which punishes ignoring rare classes) are reported instead.

9.2 Per-class breakdown



LSTM per-class F1 from the classification report. Dashed line = 0.75; `n` = test support.

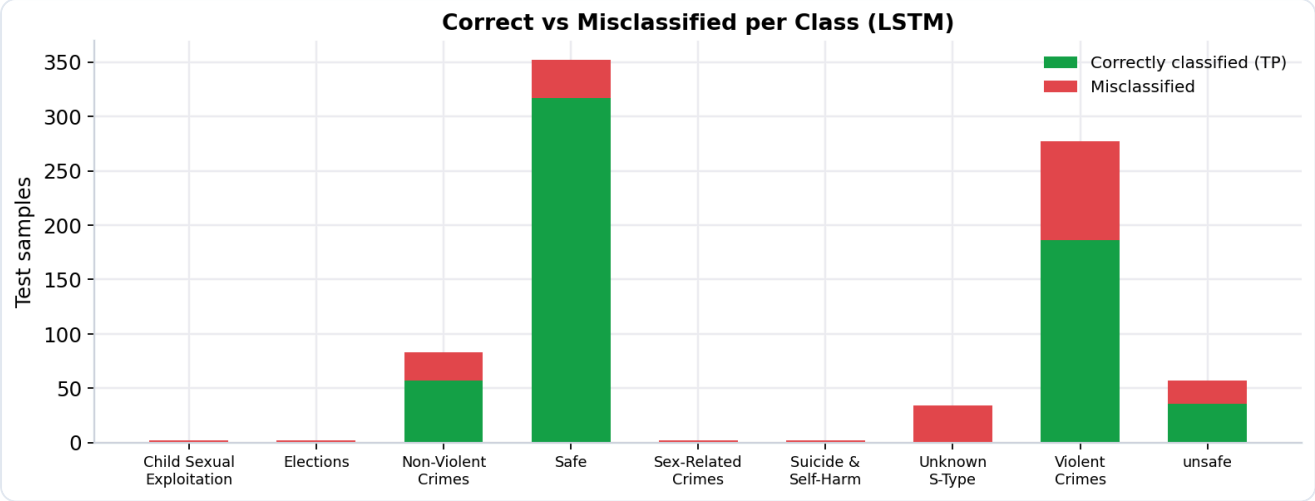
Class	Precision	Recall	F1	Support
Safe	0.76	0.90	0.82	352
Violent Crimes	0.84	0.67	0.75	277
unsafe	0.71	0.63	0.67	57
Non-Violent Crimes	0.47	0.69	0.56	83
Unknown S-Type	0.00	0.00	0.00	34
Elections / Sex-Related / Child S.E. / Suicide	0.00	0.00	0.00	2 each

Interpretation: the LSTM is competitive on the three big classes (*Safe* 0.82, *Violent* 0.75) but **scores 0.00 on five classes** — not only the 2-sample classes but also *Unknown S-Type* (34 samples), which the RNN at least partially caught. That collapse drags the macro F1 down to **0.31**, notably worse than the RNN's 0.51.

10 Confusion-Matrix Analysis

NOT PRESENT IN THE NOTEBOOK

`lstm.ipynb` does not build a confusion matrix. The chart below is a faithful proxy derived from the classification report (`recall × support` = correctly classified per class).



Correct (TP) vs misclassified test samples per class for the LSTM, derived from recall and support.

Reading the errors: *Safe* has high recall (0.90) — few safe queries are missed — but *Violent Crimes* recall is only 0.67, meaning ~1 in 3 violent queries is misrouted, mostly into the adjacent *Non-Violent/unsafe/Safe* buckets. The **most dangerous errors** for a safety system are toxic queries predicted as *Safe* (False Negatives on harm). All five minority classes are entirely absorbed into the majority classes, producing whole rows of zero True Positives.

11 Training Graphs

See §8. **Accuracy graph:** a smooth, gradual climb (healthy optimisation) with the train/val gap widening after epoch 5 (onset of over-fitting). **Loss graph:** both losses fall together early — good — but the validation loss reverses upward from ~epoch 5 while training loss keeps dropping, the canonical over-fitting signature. The convergence is stable (no wild oscillation), indicating the learning rate and batch size are well chosen.

12 Cell-by-Cell Walkthrough

Cells 0–33 are **identical** to the RNN notebook (same EDA + preprocessing); they are summarised, and the LSTM-specific cells 35–41 are detailed.

Cell	Purpose	Output & interpretation
0	Print interpreter path.	Confirms the project `venv`.
2–3	Imports; load CSV; `head()`.	3 columns load correctly.
4–6	`shape`, `info`, missing check.	3,000×3, all non-null.
7–9	Unique labels; drop 973 duplicates; value counts.	9 classes; 2,027 rows; imbalance quantified.
10	Label count-plot.	Bar chart (§3.1).
12–15	Length stats; `sample(5)`.	query median 12; combined median 21.
17	V-stack query + captions.	4,054 rows.
19–21	Define & apply `clean_text`; print before/after.	Cleaning verified.
23–26	Tokenise; vocab; texts→sequences.	Vocab 4,500; mapping works.
27	Length percentiles.	95th pct = 21 → `maxlen`.
29	Pad sequences.	(4054, 21).
31–33	Label-encode; stratified split.	3,243 train / 811 test.
35	**Build LSTM**: Embedding→LSTM(64)→Dense(32,relu)→Dropout(0.5)→Dense(9).	Deprecation warning on `input_length` (harmless). Note the added Dropout vs the RNN model.
37	Compile (adam, sparse CE).	TensorFlow GPU-unavailable-on-Windows warning — training falls back to CPU (harmless, just slower).
38	Train 10 epochs.	Log (§8): train acc 0.38→0.93, val acc peaks ~0.72 @ epoch 5, val loss rises after — over-fitting.
39	Evaluate + predict.	Test loss 1.234, test accuracy 0.735.
40	`argmax` of probabilities.	(811, 9) → hard labels.

41	Accuracy, weighted/macro F1, report.	Weighted F1 **0.717**, macro 0.310; five classes at 0.00. Ill-defined-precision warning (empty predicted classes) — expected.

13 Final Discussion

HEADLINE FINDING

On this dataset and configuration the LSTM **under-performs the SimpleRNN** (weighted F1 0.717 vs 0.753; macro F1 0.310 vs 0.506). A more powerful architecture did not translate into a better score — a valuable, honest result that points squarely at the *data*, not the model, as the bottleneck.

Why the LSTM scored lower — likely causes

- **Too little data for a gated network:** an LSTM has ~50 % more parameters and more gates to fit than a simple RNN; on ~3.2k training rows it needs more epochs / data to reach its potential.
- **Dropout(0.5) + only 10 epochs:** heavy dropout slows convergence, and training was cut at 10 epochs — the LSTM may still have been improving. It was under-trained, not over-powered.
- **Severe imbalance dominates:** with 2–5 examples per minority class, no architecture can learn them; both models fail here, and the LSTM happened to also drop *Unknown S-Type*.

Strengths

- Correct, regularised architecture (Dropout added) that trains stably with a healthy gradual learning curve.
- Solves the vanishing-gradient problem in principle and is the right stepping-stone toward BiLSTM / attention.
- Strong on the majority classes (*Safe* F1 0.82).

Recommended improvements (to reach F1 ≥ 0.85)

Direction	Expected benefit
Train longer + **early stopping** on val F1	let the LSTM converge; stop at the best epoch (~5)
Class weights / oversampling for rare classes	directly targets the macro-F1 collapse — biggest lever
Bidirectional LSTM / GRU	read context both directions; often +several F1 points
Pre-trained embeddings (GloVe/Word2Vec) or a Transformer encoder (e.g. DistilBERT)	external language knowledge; state-of-the-art on small text sets
Add an **attention** layer over LSTM outputs	focus on the toxic keyword(s) in each query

Merge / collect more minority-class data	the fundamental fix — the current 2–5 examples are unlearnable

Lessons learned

A bigger model is not automatically a better model: **data quality and balance dominate architecture** when examples are scarce. The correct next experiments are data-centric (balancing, augmentation, more data) and training-centric (early stopping, longer schedules), before reaching for ever-larger networks.

14 Assignment-Requirement Verification

	Requirement	Evidence / status
✓	LSTM model trained	Embedding→LSTM(64)→Dense(32)→Dropout→Dense(9), 10 epochs (cells 35–38)
✓	Code included	Full `lstm.ipynb` in the repository
✓	PDF report included	This document
✓	F1-score used	Weighted & macro F1 + classification report (cell 41)
✗	**F1 ≥ 0.85**	Weighted F1 = **0.717**, macro 0.310 — **threshold NOT met** (see §13 for causes & fixes)
✓	Training results discussed	§8 & §9 with over-fitting analysis
•	Bonus training graphs	Not in the notebook; reconstructed for this report (§8)
•	Confusion matrix	Not in the notebook; recall-based proxy provided (§10)

HONEST VERDICT

The LSTM was trained, evaluated with F1, and fully analysed, but its **weighted F1 of 0.717 falls short of the 0.85 target** — and below the SimpleRNN baseline. The gap is driven by extreme class imbalance plus an under-trained, heavily-regularised network stopped at 10 epochs. §13 lists the concrete, prioritised changes (class weighting, longer training with early stopping, BiLSTM/attention, more minority data) expected to close it.